



Master1 RSI

Corrigé TD 3 (Strategy /Composite)

Exercice 1

```
/* *****interface MachineComponent***** */

public interface MachineComponent {

    public void actionOn();

    public void actionOff();
}

/*****MachineComposite*****/

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import javax.swing.JComponent;
import javax.swing.JSlider;
import javax.swing.event.ChangeEvent;
import javax.swing.event.ChangeListener;

public class MachineComposite implements MachineComponent,
ActionListener, ChangeListener {
    ArrayList<MachineComponent> components = new
    ArrayList<MachineComponent>();

    JComponent component;
    boolean state = false;

    public MachineComposite(JComponent component) { this.component =
    component; }

    public void actionOn() {
        component.setEnabled(true);

        //Si l'état du JToggle button est à « appuyé » alors rendre les
        boutons dépendant accessible (enabled).

        if(state) {for(MachineComponent component : components)
        {component.actionOn(); }} } //fin de actionOn()

    public void actionOff() {
        component.setEnabled(false);
        for(MachineComponent component : components) {
            component.actionOff();
        } } //fin de actionOff()
```



```
public void actionPerformed(ActionEvent e) {
    //Inverser l'état state = !state;

    if(state) { actionOn();
    } else { actionOff();
    } } // fin de actionPerformed

public void addComponent(MachineComponent c) {
    components.add(c); }

public void removeComponent(MachineComponent c) {
    components.remove(c); }

public void stateChanged(ChangeEvent e) {
    JSlider slider = (JSlider) e.getSource();
    //Inverser l'état à partir de la valeur du JSlider state =
    (slider.getValue() != 0);
    if(state) { actionOn();
    } else { actionOff();
    }
    //Override le setEnabled false destiné aux JToggleButton

    slider.setEnabled(true); } }

/***** class MachineLeaf*****/
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JComponent;

public class MachineLeaf implements MachineComponent,
    ActionListener {
    JComponent component;
    boolean state = false;
    public MachineLeaf(JComponent component) { this.component =
    component; }
    public void actionOn() {
    component.setEnabled(true);
    }
    public void actionOff() {
    component.setEnabled(false);
    }
    public void actionPerformed(ActionEvent e) {

        //Inverser l'état state = !state;

        if(state) { actionOn();

        } else {
        actionOff(); }
        } }

/***** class PanneauControleJFrame* *****/

package exercicecomposite;

import java.util.Hashtable;

import javax.swing.JLabel;

import javax.swing.event.ChangeListener;

// la partie déclaration est à la fin de cette classe
```



```
public class PanneauContrôleJFrame extends javax.swing.JFrame {

    /** Creates new form PanneauContrôleJFrame */

    public PanneauContrôleJFrame() {

        initComponents(); // voir son code plus en bas

        // Initialisation du JSlider
        jSlider1.setInverted(true);

        Hashtable<Integer, JLabel> sliderLabels = new Hashtable<Integer, JLabel>();

        sliderLabels.put(new Integer(1), new JLabel("En marche"));

        sliderLabels.put(new Integer(0), new JLabel("À l'arrêt"));

        jSlider1.setLabelTable(sliderLabels);    jSlider1.setPaintLabels(true);

        jSlider1.setValue(0);

        *****Construction de l'arbre de composites*****

        MachineLeaf stockerComposite = new MachineLeaf(jToggleButton5);
        jToggleButton5.addActionListener(stockerComposite);
        MachineComposite empaqueterComposite = new
        MachineComposite(jToggleButton4);
        jToggleButton4.addActionListener(empaqueterComposite);
        empaqueterComposite.addComponent(stockerComposite);
        MachineLeaf jeterComposite = new
        MachineLeaf(jToggleButton7);
        jToggleButton7.addActionListener(jeterComposite);
        MachineComposite decouperComposite = new
        MachineComposite(jToggleButton2);
        jToggleButton2.addActionListener(decouperComposite);
        decouperComposite.addComponent(jeterComposite);
        decouperComposite.addComponent(empaqueterComposite);
        MachineLeaf nettoyerComposite = new
        MachineLeaf(jToggleButton6);
        jToggleButton6.addActionListener(nettoyerComposite);
        MachineComposite diagnostiqueComposite = new
        MachineComposite(jToggleButton3);
        jToggleButton3.addActionListener(diagnostiqueComposite);
        diagnostiqueComposite.addComponent(nettoyerComposite);

        MachineComposite sliderComposite = new
        MachineComposite(jSlider1);
        jSlider1.addChangeListener(sliderComposite);
        sliderComposite.addComponent(decouperComposite);
        sliderComposite.addComponent(diagnostiqueComposite);    }

        *****Cette partie concerne l'initialisation*****
        *****des composants graphiques*****

        /** This method is called from within the constructor to initialize the form.* WARNING: Do NOT modify this code. The content of this
        method is always * regenerated by the Form Editor.*/

        @SuppressWarnings("unchecked")

        // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN: initComponents

        private void initComponents() {

            jToggleButton2 = new javax.swing.JToggleButton();

            jToggleButton3 = new javax.swing.JToggleButton();

            jToggleButton4 = new javax.swing.JToggleButton();
```





```
jToggleButton5 = new javax.swing.JToggleButton();

jToggleButton6 = new javax.swing.JToggleButton(); jToggleButton7 = new javax.swing.JToggleButton(); jSlider1 = new javax.swing.JSlider();

jLabel1 = new javax.swing.JLabel(); setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

jToggleButton2.setText("Découper"); jToggleButton2.setEnabled(false);

jToggleButton3.setText("Diagnostic"); jToggleButton3.setEnabled(false);

jToggleButton4.setText("Empaqueter"); jToggleButton4.setEnabled(false);

jToggleButton5.setText("Stocker"); jToggleButton5.setEnabled(false);

jToggleButton6.setText("Nettoyer"); jToggleButton6.setEnabled(false);

jToggleButton7.setText("Jeter"); jToggleButton7.setEnabled(false);

jSlider1.setMaximum(1); jSlider1.setToolTipText("");

jLabel1.setText("État de la machine");

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);layout.setHorizontalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING) .addGroup(layout.createSequentialGroup()

        .addGap(86, 86, 86).addComponent(jToggleButton2) .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,

javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE) .addComponent(jToggleButton3)

        .addGap(74, 74, 74)) .addGroup(layout.createSequentialGroup()

        .addGap(28, 28, 28).addComponent(jToggleButton4).addGap(40, 40, 40).addComponent(jToggleButton7)

        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 71,

Short.MAX_VALUE) .addComponent(jToggleButton6)

        .addGap(94, 94, 94)) .addGroup(layout.createSequentialGroup()

        .addGap(39, 39, 39).addComponent(jToggleButton5) .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE,

Short.MAX_VALUE)) .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,

layout.createSequentialGroup() .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE,

Short.MAX_VALUE)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING) .addGroup(javax.swing.GroupLayout.Alignment.TRAILING,

layout.createSequentialGroup() .addComponent(jSlider1, javax.swing.GroupLayout.PREFERRED_SIZE,

javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE) .addGap(129, 129, 129))

        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup()

            .addComponent(jLabel1) .addGap(184, 184, 184))))

); layout.setVerticalGroup(

    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING) .addGroup(layout.createSequentialGroup()

        .addGap(16, 16, 16).addComponent(jLabel1) .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED) .addComponent(jSlider1,

javax.swing.GroupLayout.PREFERRED_SIZE,

javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE) .addGap(41, 41, 41)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE) .addComponent(jToggleButton2)

        .addComponent(jToggleButton3)) .addGap(51, 51, 51)

        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE) .addComponent(jToggleButton4)

        .addComponent(jToggleButton6)

        .addComponent(jToggleButton7)) .addGap(18, 18, 18) .addComponent(jToggleButton5) .addContainerGap(37, Short.MAX_VALUE))

);
```



```
pack();)// </editor-fold>//GEN-END:initComponents

/** @param args the command line arguments */

public static void main(String args[]) {

    /* Set the Nimbus look and feel *///<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) "> /* If Nimbus
    (introduced in Java SE 6) is not available, stay with the default look and

    feel.* For details see

    http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html */

    try {for (javax.swing.UIManager.LookAndFeelInfo info :

        javax.swing.UIManager.getInstalledLookAndFeels()) { if ("Nimbus".equals(info.getName())) {

            javax.swing.UIManager.setLookAndFeel(info.getClassName());

            break; }

        }} catch (ClassNotFoundException ex) {

            java.util.logging.Logger.getLogger(PanneauControleJFrame.class.getName()).log(java.util.lo gging.Level.SEVERE, null, ex);

        } catch (InstantiationException ex) {

            java.util.logging.Logger.getLogger(PanneauControleJFrame.class.getName()).log(java.util.lo gging.Level.SEVERE, null, ex);

        } catch (IllegalAccessException ex) {

            java.util.logging.Logger.getLogger(PanneauControleJFrame.class.getName()).log(java.util.lo gging.Level.SEVERE, null, ex);

        } catch (javax.swing.UnsupportedLookAndFeelException ex) {

            java.util.logging.Logger.getLogger(PanneauControleJFrame.class.getName()).log(java.util.lo gging.Level.SEVERE, null, ex);

        }

    }

    //</editor-fold>

    /* Create and display the form */ java.awt.EventQueue.invokeLater(new Runnable() {

        public void run() {new PanneauControleJFrame().setVisible(true);

        }

    });

    // Variables declaration - do not modify//GEN-BEGIN:variables

    private javax.swing.JLabel jLabel1;

    private javax.swing.JSlider jSlider1;

    private javax.swing.JToggleButton jToggleButton2;

    private javax.swing.JToggleButton jToggleButton3;

    private javax.swing.JToggleButton jToggleButton4;

    private javax.swing.JToggleButton jToggleButton5;

    private javax.swing.JToggleButton jToggleButton6;

    private javax.swing.JToggleButton jToggleButton7;

    // End of variables declaration//GEN-END:variables

}
```

Exrecice 2

```
public interface CheckStrategy {

    public boolean check(String s); }
```

Voici l'implémentation des 3 stratégies demandées :



```
public class StartWithT implements CheckStrategy {  
    public boolean check(String s) {  
        if( s == null || s.length() == 0) return false;  
        return s.charAt(0) == 't'; } }  
  
public class LongerThan5 implements CheckStrategy{ public  
    boolean check(String s) {  
        if(s == null) return false; return s.length() > 5; } }  
  
public class Palindrome implements CheckStrategy {  
    public boolean check(String s) {  
        if(s == null) return false;int length = s.length();  
        if(length < 2) return true;  
  
        int half = length/2;  
  
        for(int i = 0; i < half; ++i)if(s.charAt(i) !=  
            s.charAt(length - 1 - i)) return false;  
  
        return true; } }
```

La partie du programme qui utilise ces strategies:

```
public void printWhen(String filename, CheckStrategy  
which) throws IOException {  
  
    BufferedReader infile = new BufferedReader(new  
        FileReader(filename));  
  
    String buffer = null;  
  
    while((buffer = infile.readLine()) != null)  
  
    { StringTokenizer words = new StringTokenizer(buffer);  
        while( words.hasMoreTokens() )  
  
        { String word = words.nextToken();  
  
            if (which.check(word)) System.out.println(word); } } }
```

Cette méthode lit le fichier entier, une ligne à la fois, puis traite chaque ligne, un mot à la fois. Pour chaque mot, elle applique la méthode de vérification de la stratégie et imprime le mot si la stratégie renvoie vrai. Nous pouvons utiliser cette méthode pour imprimer des palindromes à partir d'un fichier avec la commande :
`printWhen("henry5.txt", new Palindrome());`



Nous pouvons rendre ce qui précède encore plus utile en paramétrant certaines des stratégies. Par exemple, si nous vérifions les mots de plus de 5 caractères, nous pourrions également vouloir vérifier les mots de plus de 3 caractères. Voici une stratégie plus générale :

```
public class LongerThanN implements CheckStrategy{  
  
    public LongerThanN(size n) {this.n = n;}  
  
    private int n;  
  
    public boolean check(String s) {  
  
        if(s == null) return false;  
  
        s.length() > n; } } }
```

Avec cette stratégie, nous pouvons maintenant trouver des mots de plus de cinq caractères avec :

```
printWhen("henry5.txt", new LongerThanN(5));
```

Il existe de nombreux types de stratégies. Ici, nous avons simplement utilisé une stratégie de vérification. Nous pourrions également employer un autre type de stratégie qui effectuerait différentes actions sur les objets en fonction de la mise en œuvre de la stratégie. Par exemple, dans tous les problèmes ci-dessus, nous devons imprimer les mots qui répondaient à un critère. Cependant, il peut y avoir plusieurs possibilités d'actions. Il pourrait s'agir de les sauvegarder dans un vecteur, de les encoder avec un chiffrement César et d'imprimer ces résultats, et bien d'autres encore. Différents types de stratégies pourraient être employés pour écrire des programmes flexibles capables de résoudre une grande classe de problèmes, plutôt qu'un seul.

Exercice 3

Nous pouvons concevoir un décorateur pour les stratégies (puisque les stratégies sont définies par une interface). Un décorateur de stratégies va prendre comme paramètre dans son constructeur une stratégie et va lui-même implémenter l'interface stratégie.

```
class CounterDecorator implements CheckStrategy {  
  
    private CheckStrategy checker;  
  
    public CounterDecorator(CheckStrategy check) {  
  
        checker = check; } }
```



class CounterDecorator implements
CheckStrategy
{ public CounterDecorator
(CheckStrategy checker)
{ checker = check;}
public boolean check(String s)
{ boolean result = checker.check(s);
if(result)count++;
return result;
}
public int count(){return count;}
public void reset(){count = 0;}
private int count = 0;
private CheckStrategy checker;
}

```
public void reset(){count = 0;}

private int count = 0; }
```

Maintenant, on peut trouver les palindromes, les afficher et les comptabiliser avec ce qui suit:

```
CounterDecorator counter = new CounterDecorator (new
Palindrome());

printWhen("henry5.txt", counter);

System.out.println("'" + counter.count());
```

Remarquez que la méthode de vérification du décorateur count appelle la méthode de vérification de la méthode palindrome avec laquelle elle est initialisée. Elle compte alors le mot si le résultat est vrai. Notez que ni la classe palindrome ni la méthode **printWhen** n'ont dû être modifiées pour obtenir ce résultat. C'est la puissance du patron décorateur. Notez également que les décorateurs peuvent fournir des méthodes supplémentaires au-delà de ce qui est requis par l'interface qu'ils décorent. Dans cet exemple, nous avons les méthodes count et reset du CounterDecorator.

Exercice 4

Nous allons construire un composite strategy, qui nous permet de composer des strategies en utilisant le AND logique

```
class AndStrategyComposite implements CheckStrategy {

private java.util.Vector tests = new Vector();

public void addStrategy(CheckStrategy s) {

    tests.addElement(s); }

public boolean check(String s)

{ java.util.Enumeration e = tests.elements();

while (e.hasMoreElements()){

    CheckStrategy strategy = (CheckStrategy)e.nextElement();

    if( ! strategy.check(s) ) return false; return true; }

}
```

Cette stratégie retourne false si n'importe quelle des stratégies que nous lui avons ajoutées retourne false pour la même chaîne de caractère. Sinon elle retourne true.



Remarque, qu'elle retourne true, si nous n'avons ajouté aucune stratégie.

Nous pouvons maintenant créer un nouveau AndStrategyComposite et lui ajouter d'autres stratégies. Nous pouvons utiliser ce nouvel objet comme une stratégie.

```
... AndStrategyComposite longWordsThatStartWithT = new  
AndStrategyComposite();  
longWordsThatStartWithT.addStrategy(new LongerThan(5));  
longWordsThatStartWithT.addStrategy(new StartWithT()); ...  
println("henry5.txt", longWordsThatStartWithT);
```

Et bien sûr, puisqu'il ne s'agit que d'une stratégie, nous pourrions la décorer d'un compteur et obtenir également le décompte.